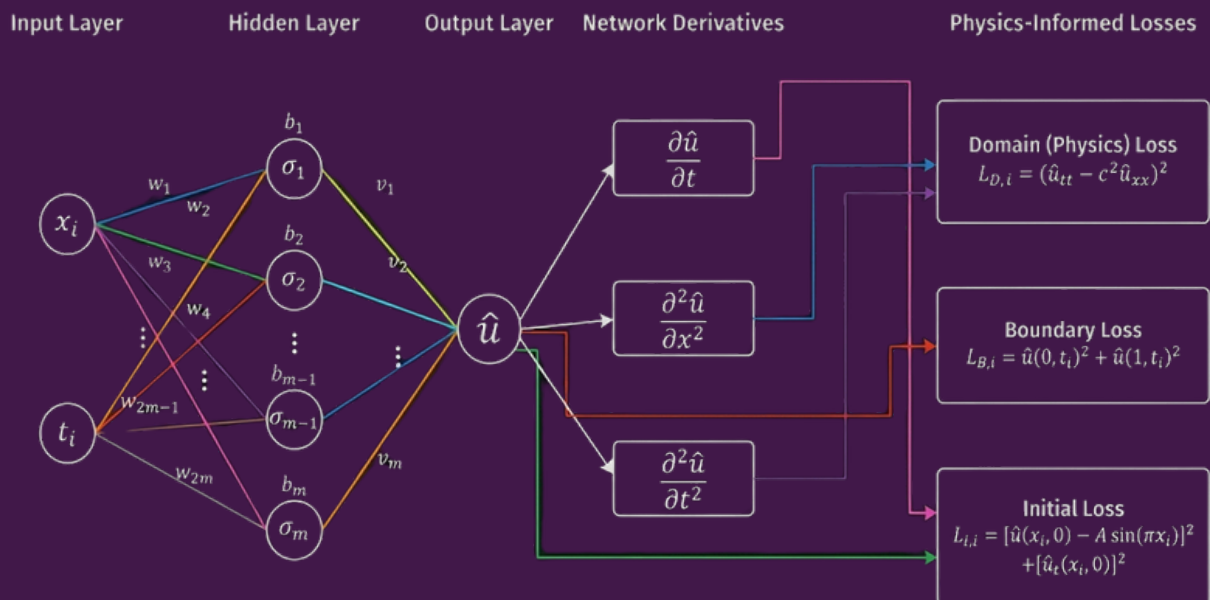


Modules for Machine and Deep Learning

Module 5: Part A – Physics-Informed Neural Network (PINN) from Scratch (Unscaled Single-Hidden-Layer Network)



Assoc. Prof. Ir. Dr. Airil Yasreen Mohd Yassin

School of Energy, Geoscience, Infrastructure and Society, Heriot-Watt University Malaysia

Assoc. Prof. Ts. Dr. Ahmad Razin Zainal Abidin

Faculty of Civil Engineering, Universiti Teknologi Malaysia

Dr. Halinawati Hirol

Malaysia-Japan International Institute of Technology (MJIT), Universiti Teknologi Malaysia

Dr. Mokhtazul Haizad Mokhtaram

Faculty of Engineering and Life Sciences, Universiti Selangor

Dr. Mohd Al-Akhbar Mohd Noor

Faculty of Engineering and Life Sciences, Universiti Selangor

Draft version
Not for distribution without
authors' permission

Table of Contents

	Page
1. Introduction	1
2. Forward Pass of an Unscaled Single-Hidden-Layer PINN	2
2.1 FFNN of $\mathbf{u}$ (NETWORK A)	3
2.2 First derivative of the FFNN with respect to $\mathbf{t}, \frac{\partial \hat{u}}{\partial t}$	4
2.3 Second derivative of the FFNN with respect to $\mathbf{x}, \frac{\partial^2 \hat{u}}{\partial x^2}$	4
2.4 Second derivative of the FFNN with respect to $\mathbf{t}, \frac{\partial^2 \hat{u}}{\partial t^2}$	6
2.5 Collocation and expansion of the network to $\mathbf{m}$ neurons	6
2.6 Residuals and Loss functions	9
2.7 Common graphical representation of PINN	11
3. Backward Pass (or Backpropagation) of a Single Hidden Layer PINN	12
3.1 Parameters update from point-losses	12
3.2 Calculation of the gradients	13
4. Python Code	16
4.1 Plotting and verification of results	21
Appendix	25

1. Introduction

In the previous modules, we discussed what are described as data-driven neural network models. In these models, we rely on sets of input and output data to train the network before making inferences. These paired or labelled data are core to the training process, and there must be sufficient amounts of them (usually a lot), hence the term data-driven associated with these models.

There are two observations we can make regarding these data-driven models:

- i. We need a lot of data. In the case where data are scarce, neural networks, as well as other data-driven machine learning models, may find themselves in a limiting situation.
- ii. There is maths involved in the network and its training process, but there is no physics explicitly considered. In the practical world, especially in science and engineering, the absence of physics from the calculation might hint at some “room for improvement”, where the inclusion or consideration of physics might not only help compensate when data are scarce, but may also provide additional information to guide the training and improve the performance of the network.

This is where Physics-Informed Neural Network (PINN) comes into the picture.

1.1 How are we going to proceed with the discussion of this module?

PINN is relatively mathematics-heavy, especially because it involves the representation of the governing equations by the networks. In lieu, it requires not only gradients with respect to the parameters (weights and biases), but also derivatives with respect to the independent variables (i.e., the spatial and time dimensions). On top of that, unlike the traditional neural networks discussed in the previous modules, PINN does not necessarily require labelled solution data. Instead, the errors associated with the governing equation, boundary conditions and initial conditions are calculated based on the residuals evaluated at the collocation points.

Scaling also requires particular attention in PINN. Since the governing equations involve derivatives with respect to the independent variables, the variables cannot simply be normalized without considering their effects on the governing equations. In this module, we will therefore discuss two scaling mechanisms:

- i. nondimensionalizing the governing equations
- ii. scaling/normalizing the individual loss components before forming the total loss.

Due to these complexities, this module is arranged in two parts, Part A and Part B.

In Part A, we will really derive the PINN from scratch, including writing the code using Python instead of PyTorch (no deep learning libraries will be used). Therefore, to keep the “maths” to a minimum, we will work with the dimensional governing equation (1D wave PDE), solved using a single-hidden-layer network. For the backpropagation, we will employ an SGD-like minimization approach, where the parameters are updated immediately for each running collocation point. Thus, we will work with the individual residual and loss associated with the particular collocation point rather than forming a total loss over all collocation points. We term the network formulation in PART A the unscaled single-hidden-layer PINN.

In Part B, we will increase the complexity of the formulation. Instead of deriving and writing everything from scratch, we will employ PyTorch and its deep learning libraries to assist us in developing a more realistic and practical PINN. We will work with the nondimensional governing equation, solved using a multi-hidden-layer network. For the calculation of the loss, we will consider the total loss and its scaling mechanisms, including normalization with or without weighting parameters. We will also employ PyTorch's automatic differentiation (autograd) to calculate the required derivatives, either with respect to the network parameters or the independent variables.

PART A

2. Forward Pass of an Unscaled Single-Hidden-Layer PINN

The basic idea of PINN is to represent the derivative terms in a governing equation (PDE or ODE) by the corresponding derivatives of the network. These derivatives are then used to construct the governing-equation residual, from which the physics loss is calculated. This loss is later summed together with the losses associated with the boundary and initial conditions to form the total loss. It is this total loss that will be minimized during the training of the network.

Consider the 1D wave equation below (domain equation):

$$\frac{\partial^2 u}{\partial t^2} - c^2 \frac{\partial^2 u}{\partial x^2} = 0 \quad (1)$$

with the following boundary and initial conditions:

Boundary conditions

$$u(0, t) = 0, u(1, t) = 0 \quad (2a)$$

Initial conditions

$$u(x, 0) = \sin(\pi x), \frac{\partial u}{\partial t}(x, 0) = 0 \quad (2b)$$

where u is the wave displacement, c is the wave propagation speed, x is the spatial coordinate and t is time.

Note that, in Eq. (2a), the length of the domain is taken as unity, and the amplitude of the sinusoidal initial condition in Eq. (2b) is also taken as unity. This is because, as mentioned in Section 1.1, since we are not employing any scaling in Part A, it is important to control the magnitude of the variables and the resulting solution, and consequently the residuals and losses, in such a way.

From the governing equations above, we can see that we need:

- i. A FFNN of u to represent the boundary conditions in Eq. (2a) and the first initial condition in Eq. (2b).
- ii. A first derivative of the FFNN with respect to t to represent the second initial condition (i.e., $\frac{\partial u}{\partial t}$) in Eq. (2b)

- iii. A second derivative of the FFNN with respect to x to represent the second term (i.e., $\frac{\partial^2 u}{\partial x^2}$) in Eq. (1)
- iv. A second derivative of the FFNN with respect to t to represent the first term (i.e., $\frac{\partial^2 u}{\partial t^2}$) in Eq. (1)

Now, let's derive the above network derivatives. Note that this is the part mentioned in Section 1.1 where, unlike traditional NNs which require gradients with respect to the parameters (weights and biases) only, PINN also requires the derivatives of the network with respect to the independent variables (i.e., x and t).

2.1 FFNN of u (NETWORK A)

To ease our expansion and derivation, let's consider a simple network having two inputs, a single hidden layer with two neurons, and an output as shown in Figure 1. We denote this network as Network A.

Such a simple architecture is chosen so that we can easily trace the mathematical operations (for forward and backward passes). Once the fundamentals are understood, the extension to more complex networks (more hidden layers and neurons per layer) will be straightforward.

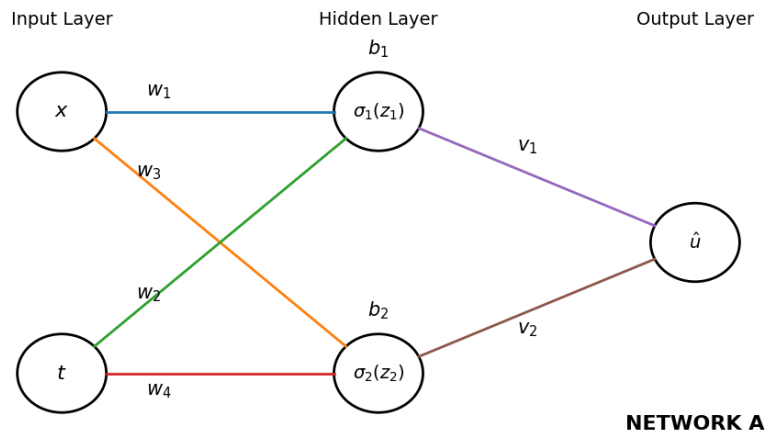


Figure 1: FFNN of u (NETWORK A)

Because the network is simple, we can write the forward pass process in expanded forms (hence our intention in the first place) as follows:

The summing variables, z_1 and z_2 :

$$\begin{aligned} z_1 &= w_1 x + w_2 t + b_1 \\ z_2 &= w_3 x + w_4 t + b_2 \end{aligned} \tag{3}$$

The activated outputs, σ_1 and σ_2 (we use tanh as the activation function):

$$\begin{aligned} \sigma_1 &= \tanh(z_1) \\ \sigma_2 &= \tanh(z_2) \end{aligned} \tag{4}$$

Thus, the output,

$$\hat{u} = v_1\sigma_1 + v_2\sigma_2 \quad (5)$$

2.2 First derivative of the FFNN with respect to t , $\frac{\partial \hat{u}}{\partial t}$

We can obtain the first derivative of Network A with respect to t using the chain rule as follows:

$$\frac{\partial \hat{u}}{\partial t} = \frac{\partial \hat{u}}{\partial \sigma_1} \frac{\partial \sigma_1}{\partial z_1} \frac{\partial z_1}{\partial t} + \frac{\partial \hat{u}}{\partial \sigma_2} \frac{\partial \sigma_2}{\partial z_2} \frac{\partial z_2}{\partial t} \quad (6)$$

Substituting Eqs. (3), (4) and (5) into Eq. (6), we then obtain:

$$\frac{\partial \hat{u}}{\partial t} = v_1(1 - \tanh^2(z_1))w_2 + v_2(1 - \tanh^2(z_2))w_4 \quad (7)$$

Eq. (7) can be given in graphical network representation, as shown in Figure 2 (i.e., $\frac{\partial \sigma_i}{\partial z_i} = 1 - \tanh^2(z_i)$). We denote this network as Network B.

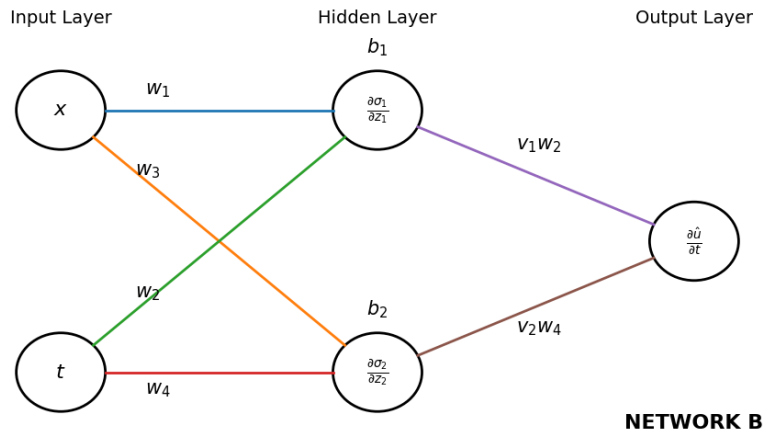


Figure 2: First derivative of the FFNN with respect to t (NETWORK B)

Note that NETWORK B is not a new network but the differentiated form of the original network, NETWORK A. Thus, the weights ($w_1, w_2, w_3, w_4, v_1, v_2$) and biases (b_1, b_2) remain the same. The same concept applies to the higher-order derivatives of the network discussed later.

2.3 Second derivative of the FFNN with respect to x , $\frac{\partial^2 \hat{u}}{\partial x^2}$

To derive the second derivative of the network with respect to x , we must first obtain its derivative with respect to x , which, using chain rule, can be given as:

$$\frac{\partial \hat{u}}{\partial x} = \frac{\partial \hat{u}}{\partial \sigma_1} \frac{\partial \sigma_1}{\partial z_1} \frac{\partial z_1}{\partial x} + \frac{\partial \hat{u}}{\partial \sigma_2} \frac{\partial \sigma_2}{\partial z_2} \frac{\partial z_2}{\partial x} \quad (8)$$

Substituting Eqs. (3), (4) and (5) into Eq. (8), we then obtain:

$$\frac{\partial \hat{u}}{\partial x} = v_1(1 - \tanh^2(z_1))w_1 + v_2(1 - \tanh^2(z_2))w_3 \quad (9)$$

Recall that $\frac{\partial \sigma_i}{\partial z_i} = 1 - \tanh^2(z_i)$, Eq. (9) can be expressed as:

$$\frac{\partial \hat{u}}{\partial x} = v_1 w_1 \frac{\partial \sigma_1}{\partial z_1} + v_2 w_3 \frac{\partial \sigma_2}{\partial z_2} \quad (10)$$

Now differentiate Eq. (10) once again to obtain the desired second derivative. Note that v_1, v_2, w_1, w_3 are constants thus will not be involved in the differentiation:

$$\frac{\partial^2 \hat{u}}{\partial x^2} = v_1 w_1 \frac{\partial}{\partial x} \left(\frac{\partial \sigma_1}{\partial z_1} \right) + v_2 w_3 \frac{\partial}{\partial x} \left(\frac{\partial \sigma_2}{\partial z_2} \right) \quad (11)$$

Employing chain rule again to the derivative terms on the RHS of Eq. (11):

$$\frac{\partial^2 \hat{u}}{\partial x^2} = v_1 w_1 \frac{\partial^2 \sigma_1}{\partial z_1^2} \frac{\partial z_1}{\partial x} + v_2 w_3 \frac{\partial^2 \sigma_2}{\partial z_2^2} \frac{\partial z_2}{\partial x} \quad (12)$$

Substituting Eq. (3) into Eq. (12), we then obtain:

$$\frac{\partial^2 \hat{u}}{\partial x^2} = v_1 w_1^2 \frac{\partial^2 \sigma_1}{\partial z_1^2} + v_2 w_3^2 \frac{\partial^2 \sigma_2}{\partial z_2^2} \quad (13)$$

Eq. (13) can be given in a graphical network representation, as shown in Figure 3. We denote this network as Network C. Note that:

- i. $\frac{\partial^2 \sigma_i}{\partial z_i^2} = -2 \tanh(z_i) (1 - \tanh^2(z_i))$
- ii. NETWORK C is not a new network but the twice-differentiated form of the original network, NETWORK A.

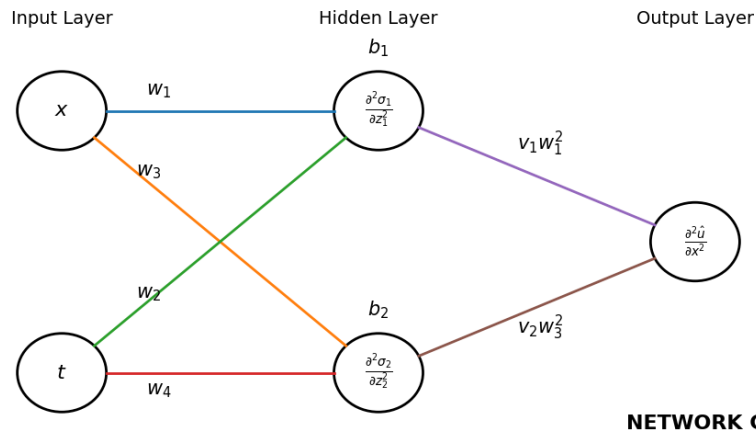


Figure 3: Second derivative of the FFNN with respect to x (NETWORK C)

2.4 Second derivative of the FFNN with respect to t , $\frac{\partial^2 \hat{u}}{\partial t^2}$

To derive the second derivative of the network with respect to t , we must first obtain its derivative with respect to t , which we already derived as Eq. (6) and (7).

Now differentiate Eq. (7) once again to obtain the desired second derivative.

$$\frac{\partial^2 \hat{u}}{\partial t^2} = v_1 w_2 \frac{\partial}{\partial t} \left(\frac{\partial \sigma_1}{\partial z_1} \right) + v_2 w_4 \frac{\partial}{\partial t} \left(\frac{\partial \sigma_2}{\partial z_2} \right) \quad (14)$$

Employing the chain rule again to the derivative terms on the RHS of Eq. (14):

$$\frac{\partial^2 \hat{u}}{\partial t^2} = v_1 w_2 \frac{\partial^2 \sigma_1}{\partial z_1^2} \frac{\partial z_1}{\partial t} + v_2 w_4 \frac{\partial^2 \sigma_2}{\partial z_2^2} \frac{\partial z_2}{\partial t} \quad (15)$$

Substituting Eq. (3) into Eq. (15), we then obtain:

$$\frac{\partial^2 \hat{u}}{\partial t^2} = v_1 w_2^2 \frac{\partial^2 \sigma_1}{\partial z_1^2} + v_2 w_4^2 \frac{\partial^2 \sigma_2}{\partial z_2^2} \quad (16)$$

Eq. (16) can be given in a graphical network representation, as shown in Figure 4. We denote this network as Network D.

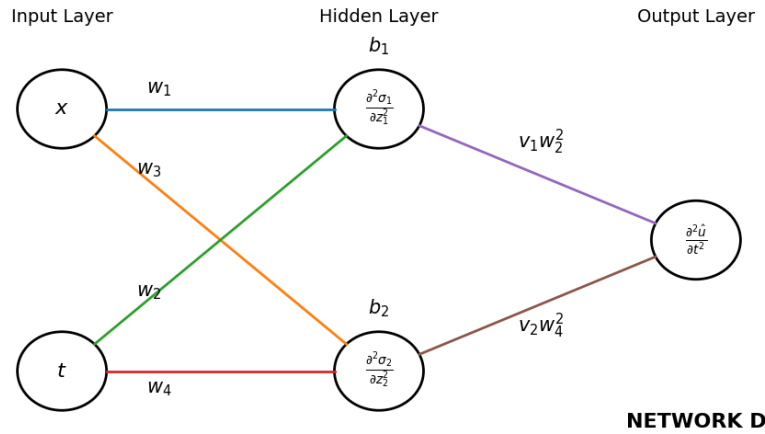


Figure 4: Second derivative of the FFNN with respect to t (NETWORK D)

2.5 Collocation and expansion of the network to m neurons

In the previous sections, we treated the spatial variable, x , and time, t , as continuous. In the PINN implementation, we must evaluate:

- the second derivative networks (i.e. NETWORKS C and D) at collocation points (x_i, t_i) in the domain interior
- the field variable, u (i.e. NETWORK A) at boundary points x_0 and x_1 (with t_i running)
- the field variable u (i.e. NETWORK A) and the first derivative network (i.e., NETWORK B) at initial time, t_0 (with x_i running)

This is to prepare for the collocation process in calculating the loss function.

Also, in the previous sections, we have used the simple 2-neuron network to ease the tracing of the derivations. Now that we have established the formulation, let's expand the single hidden layer network to m neurons.

Evaluation and expansion of Network A

The discretized and expanded form of Network A (denoted as NETWORK A (x_i, t_i)) is shown in Figure 5. The mathematical expression is given (from Eq. (5)) as:

$$\hat{u}(x_i, t_i) = \sum_{j=1}^m v_j \sigma_j(z_j(x_i, t_i)) \quad (17)$$

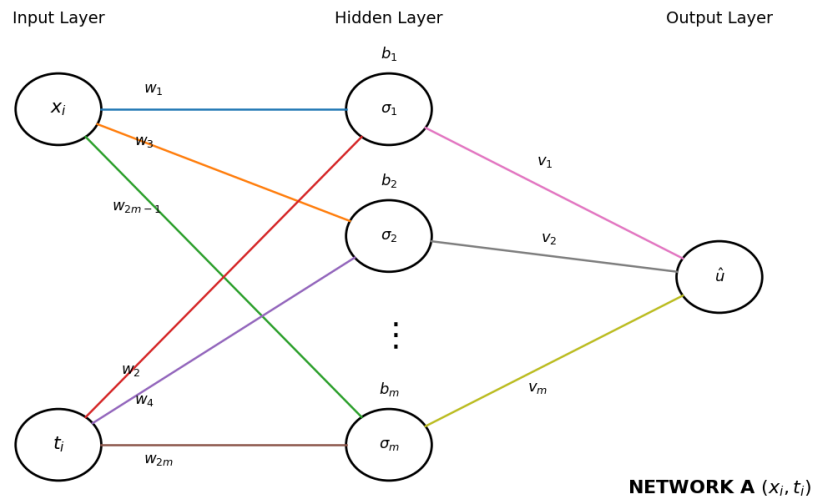


Figure 5: Evaluated and expanded form of Network A

Evaluation and expansion of Network B

The discretized and expanded form of Network B (denoted as NETWORK B (x_i, t_i)) is shown in Figure 6. The mathematical expression is given (from Eq. (7)) as:

$$\frac{\partial \hat{u}}{\partial t}(x_i, t_i) = \sum_{j=1}^m v_j w_{2j} \frac{\partial \sigma_j(z_j(x_i, t_i))}{\partial z_j} \quad (18)$$

where $2j$ means $2 \times j$.

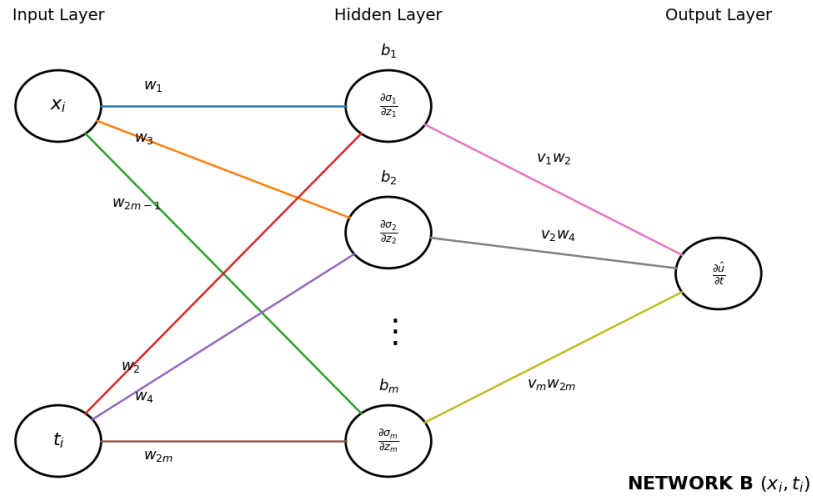


Figure 6: Evaluated and expanded form of Network B

Evaluation and expansion of Network C

The discretized and expanded form of Network C (denoted as NETWORK C (x_i, t_i)) is shown in Figure 7. The mathematical expression is given (from Eq. (13)) as:

$$\frac{\partial^2 \hat{u}}{\partial x^2}(x_i, t_i) = \sum_{j=1}^m v_j w_{2j-1}^2 \frac{\partial^2 \sigma_j(z_j(x_i, t_i))}{\partial z_j^2} \quad (19)$$

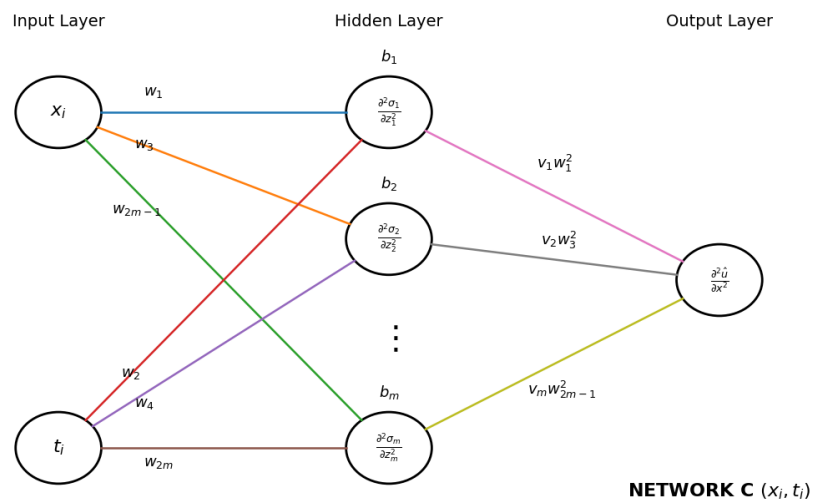


Figure 7: Evaluated and expanded form of Network C

Evaluation and expansion of Network D

The discretized and expanded form of Network D (denoted as NETWORK D (x_i, t_i)) is shown in Figure 8. The mathematical expression is given (from Eq. (16)) as:

$$\frac{\partial^2 \hat{u}}{\partial t^2}(x_i, t_i) = \sum_{j=1}^m v_j w_{2j}^2 \frac{\partial^2 \sigma_j(z_j(x_i, t_i))}{\partial z_j^2} \quad (20)$$

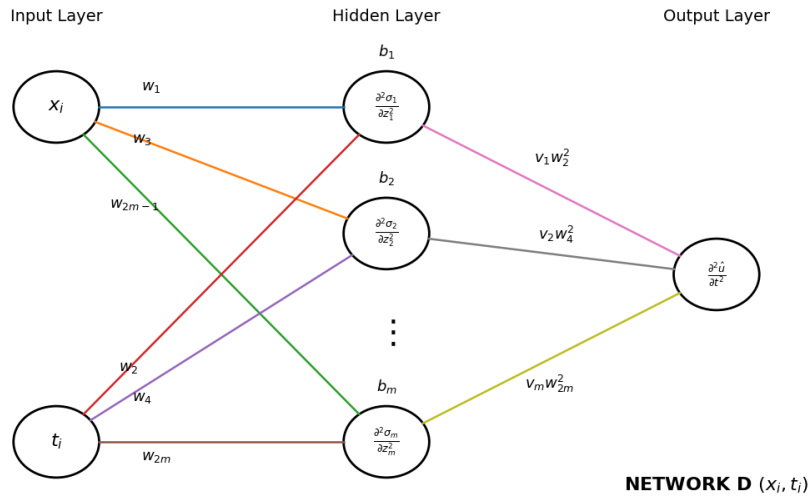


Figure 8: Evaluated and expanded form of Network D

2.6 Residuals and Loss functions

PINN differs from the traditional neural networks (described in the previous modules) in the sense that its errors arise from the nonsatisfaction of the domain equation, boundary conditions and/or initial conditions (i.e., Eqs. (1) and (2)). Using FEM terminology, these errors are called residuals, obtained from collocation.

2.6.1 Domain residual, $R_{D,i}$ and domain loss, $L_{D,i}$

Nonsatisfaction of Eq. (1) gives rise to domain residual, $R_{D,i}$, given as:

$$R_{D,i} = \frac{\partial^2 \hat{u}}{\partial t^2} - c^2 \frac{\partial^2 \hat{u}}{\partial x^2} \quad (21)$$

where $R_{D,i}$ is the residual at the collocation point (x_i, t_i) and is zero only if the domain equation is completely satisfied.

With the derivations of the network and its derivatives discussed in Section 2.1 to 2.4, Eq. (21) can equivalently be represented as:

In Networks C and D representation

$$R_{D,i} = \text{NETWORK D}(x_i, t_i) - c^2(\text{NETWORK C})(x_i, t_i) \quad (22)$$

In expanded equation representation (from Eqs. (19) and (20))

$$R_{D,i} = \sum_{j=1}^m v_j w_{2j}^2 \frac{\partial^2 \sigma_j(z_j(x_i, t_i))}{\partial z_j^2} - c^2 \sum_{j=1}^m v_j w_{2j-1}^2 \frac{\partial^2 \sigma_j(z_j(x_i, t_i))}{\partial z_j^2} \quad (23)$$

The domain loss at the collocation point of (x_i, t_i) , $L_{D,i}$ is thus:

$$L_{D,i} = R_{D,i}^2 \quad (24)$$

2.6.2 Boundary residuals, $R_{B,i}$ and boundary loss, $L_{B,i}$

Nonsatisfaction of Eq. (2a) gives rise to boundary residuals, $R_{B,i}$, given as:

$$\begin{aligned} R_{B0,i} &= \hat{u}(0, t_i) \\ R_{BL,i} &= \hat{u}(L, t_i) \end{aligned} \quad (25)$$

where $R_{B0,i}$ is the residual at the boundary point $(0, t_i)$ and $R_{BL,i}$ is the residual at the boundary point (L, t_i) . Both are zero only if the evaluated value is equal to the prescribed value.

With the derivations of the network and its derivatives discussed in Section 2.1 to 2.4, Eq. (25) can equivalently be represented as:

In Networks A representation

$$\begin{aligned} R_{B0,i} &= \text{NETWORK A}(0, t_i) \\ R_{BL,i} &= \text{NETWORK A}(L, t_i) \end{aligned} \quad (26)$$

In expanded equation representation (from Eqs. (17))

$$\begin{aligned} R_{B0,i} &= \sum_{j=1}^m v_j \sigma_j(z_j(0, t_i)) \\ R_{BL,i} &= \sum_{j=1}^m v_j \sigma_j(z_j(L, t_i)) \end{aligned} \quad (27)$$

The boundary loss at the boundary points of $(0, t_i)$ and (L, t_i) , $L_{B,i}$ is thus:

$$L_{B,i} = R_{B0,i}^2 + R_{BL,i}^2 \quad (28)$$

2.6.3 Initial condition residuals, $R_{I,i}$ and initial condition loss, $L_{I,i}$

Nonsatisfaction of Eq. (2b) gives rise to initial condition residuals, $R_{I,i}$, given as:

$$R_{I,i} = \hat{u}(x_i, 0) - \sin(\pi x_i) \quad (29)$$

$$R_{\partial I,i} = \frac{\partial \hat{u}}{\partial t}(x_i, 0)$$

where $R_{I,i}$ is the residual of the field variable $\hat{u}$ at $t=0$, and $R_{\partial I,i}$ is the residual of the initial velocity at $t=0$. Both are zero only if the evaluated values are equal to the prescribed values.

With the derivations of the network and its derivatives discussed in Section 2.1 to 2.4, Eq. (29) can equivalently be represented as:

In Networks A and B representation

$$R_{I,i} = \text{NETWORK A}(x_i, 0) - \sin(\pi x_i) \quad (30)$$

$$R_{\partial I,i} = \text{NETWORK B}(x_i, 0)$$

In expanded equation representation (from Eqs. (17) and (18))

$$R_{I,i} = \sum_{j=1}^m v_j \sigma_j(z_j(x_i, 0)) - \sin(\pi x_i) \quad (31)$$

$$R_{\partial I,i} = \sum_{j=1}^m v_j w_{2j} \frac{\partial \sigma_j(z_j(x_i, 0))}{\partial z_j}$$

The initial condition loss and initial time $t=0$, $L_{I,i}$ is thus:

$$L_{I,i} = R_{I,i}^2 + R_{\partial I,i}^2 \quad (32)$$

2.7 Common graphical representation of PINN

Figures 2 to 4 (and Figures 6 to 8, for that matter), and the use of NETWORKS to represent the governing equations, are our own representation. We introduced this approach because we believe that it can give a clearer meaning to what a derivative means in a neural network; that is, it is the derivative of the network itself.

However, the standard graphical representation of PINN commonly found in the literature is as given in Figure 9. Therefore, it is also presented herein so that readers can become familiar with both representations.

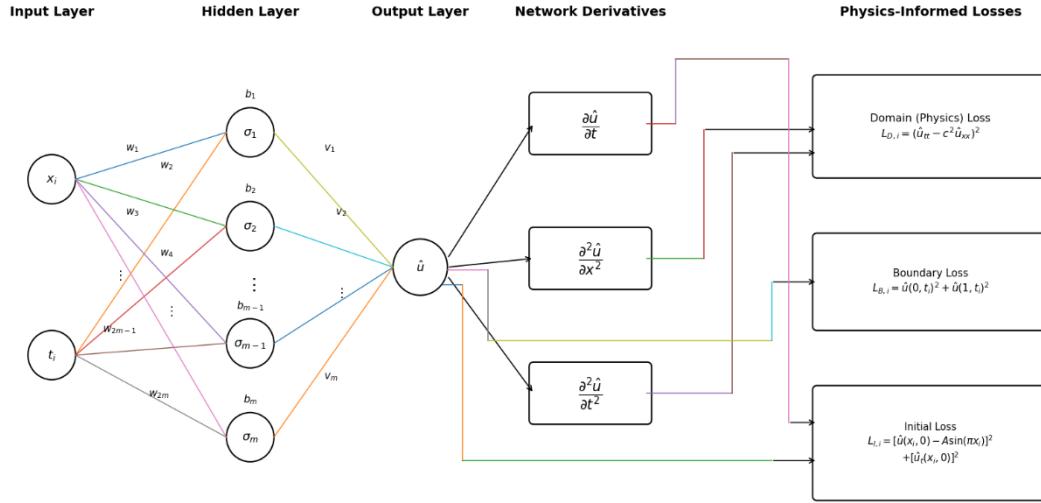


Figure 9: Common graphical representation of a PINN

3. Backward Pass (or Backpropagation) of a Single Hidden Layer PINN

With the establishment of the point-losses $L_{D,i}$, $L_{B,i}$ and $L_{I,i}$, we are now in the position to calculate their gradients by backpropagation and subsequently update the network parameters to minimize the loss. As mentioned in Section 1.1, we are able to do this because, for Part A, we are employing an SGD-like minimization where the updating is carried out immediately after the running of one collocation point (or boundary or initial point, for that matter). In other words, we are not working with the total loss for now.

Also, since we want to derive and write everything from scratch using Python, we will explicitly consider the parameter updates for each case (domain, boundary and initial conditions). Thus, our subsequent discussion will be a bit lengthy and repetitive. Readers who have already grasped the concept may therefore skip the repetitive derivations. However, in Part B, things will become more practical and “economical” when we employ PyTorch to do much of the heavy lifting.

3.1 Parameters update from point-losses

Since we are employing SGD-like minimization, our parameters will be updated by each of the point losses (not the total loss). But take note that, in the following parameter updates, the subscript k denotes the neuron whose parameters are being updated, where $k=1,2,\dots,m$. This is distinguished from the j , which is used in the earlier equations as the summation index.

- i. The parameter update from domain point-loss, $L_{D,i}$

$$v_{k,new} = v_{k,old} - \mu \frac{\partial L_{D,i}}{\partial v_k} \quad (33)$$

$$b_{k,new} = b_{k,old} - \mu \frac{\partial L_{D,i}}{\partial b_k} \quad (34)$$

$$w_{2k-1,new} = w_{2k-1,old} - \mu \frac{\partial L_{D,i}}{\partial w_{2k-1}} \quad (35)$$

$$w_{2k,new} = w_{2k,old} - \mu \frac{\partial L_{D,i}}{\partial w_{2k}} \quad (36)$$

ii. Parameter update from boundary loss, $L_{B,i}$

$$v_{k,new} = v_{k,old} - \mu \frac{\partial L_{B,i}}{\partial v_k} \quad (37)$$

$$b_{k,new} = b_{k,old} - \mu \frac{\partial L_{B,i}}{\partial b_k} \quad (38)$$

$$w_{2k-1,new} = w_{2k-1,old} - \mu \frac{\partial L_{B,i}}{\partial w_{2k-1}} \quad (39)$$

$$w_{2k,new} = w_{2k,old} - \mu \frac{\partial L_{B,i}}{\partial w_{2k}} \quad (40)$$

iii. Parameter update from initial condition loss, $L_{I,i}$

$$v_{k,new} = v_{k,old} - \mu \frac{\partial L_{I,i}}{\partial v_k} \quad (41)$$

$$b_{k,new} = b_{k,old} - \mu \frac{\partial L_{I,i}}{\partial b_k} \quad (42)$$

$$w_{2k-1,new} = w_{2k-1,old} - \mu \frac{\partial L_{I,i}}{\partial w_{2k-1}} \quad (43)$$

$$w_{2k,new} = w_{2k,old} - \mu \frac{\partial L_{I,i}}{\partial w_{2k}} \quad (44)$$

3.2 Calculation of the gradients

From Eqs. (33) to (44), for the parameter updates, we can see that we need to calculate 12 set of gradients. Once these gradients are established, the process is complete and we are ready to write the Python code. So, let's establish the gradients.

i. Calculation of $\frac{\partial L_{D,i}}{\partial v_k}$

From Eq. (23) and (24):

$$\frac{\partial L_{D,i}}{\partial v_k} = 2R_{D,i}(w_{2k}^2 - c^2 w_{2k-1}^2) \frac{\partial^2 \sigma_k(z_k(x_i, t_i))}{\partial z_k^2} \quad (45)$$

ii. Calculation of $\frac{\partial L_{D,i}}{\partial b_k}$

From Eq. (23) and (24):

$$\frac{\partial L_{D,i}}{\partial b_k} = 2R_{D,i}v_k(w_{2k}^2 - c^2w_{2k-1}^2) \frac{\partial^3 \sigma_k(z_k(x_i, t_i))}{\partial z_k^3} \quad (46)$$

iii. Calculation of $\frac{\partial L_{D,i}}{\partial w_{2k-1}}$

From Eq. (23) and (24):

$$\begin{aligned} \frac{\partial L_{D,i}}{\partial w_{2k-1}} = 2R_{D,i}v_k \left[-2c^2w_{2k-1}^2 \left(\frac{\partial^2 \sigma_k(z_k(x_i, t_i))}{\partial z_k^2} \right) \right. \\ \left. + x_i(w_{2k}^2 - c^2w_{2k-1}^2) \left(\frac{\partial^3 \sigma_k(z_k(x_i, t_i))}{\partial z_k^3} \right) \right] \end{aligned} \quad (47)$$

iv. Calculation of $\frac{\partial L_{D,i}}{\partial w_{2k}}$

From Eq. (23) and (24):

$$\begin{aligned} \frac{\partial L_{D,i}}{\partial w_{2k}} = 2R_{D,i}v_k \left[2w_{2k}^2 \left(\frac{\partial^2 \sigma_k(z_k(x_i, t_i))}{\partial z_k^2} \right) \right. \\ \left. + t_i(w_{2k}^2 - c^2w_{2k-1}^2) \left(\frac{\partial^3 \sigma_k(z_k(x_i, t_i))}{\partial z_k^3} \right) \right] \end{aligned} \quad (48)$$

v. Calculation of $\frac{\partial L_{B,i}}{\partial v_k}$

From Eq. (27) and (28):

$$\frac{\partial L_{B,i}}{\partial v_k} = 2R_{B0,i}\sigma_k(z_k(0, t_i)) + 2R_{BL,i}\sigma_k(z_k(L, t_i)) \quad (49)$$

vi. Calculation of $\frac{\partial L_{B,i}}{\partial b_k}$

From Eq. (27) and (28):

$$\frac{\partial L_{B,i}}{\partial b_k} = 2v_k \left[R_{B0,i} \frac{\partial \sigma_k(z_k(0, t_i))}{\partial z_k} + R_{BL,i} \frac{\partial \sigma_k(z_k(L, t_i))}{\partial z_k} \right] \quad (50)$$

vii. Calculation of $\frac{\partial L_{B,i}}{\partial w_{2k-1}}$

From Eq. (27) and (28):

$$\frac{\partial L_{B,i}}{\partial w_{2k-1}} = 2R_{BL,i}v_kL \frac{\partial \sigma_k(z_k(L, t_i))}{\partial z_k} \quad (51)$$

viii. Calculation of $\frac{\partial L_{B,i}}{\partial w_{2k}}$

From Eq. (27) and (28):

$$\frac{\partial L_{B,i}}{\partial w_{2k}} = 2v_k t_i \left[R_{B0,i} \frac{\partial \sigma_k(z_k(0, t_i))}{\partial z_k} + R_{BL,i} \frac{\partial \sigma_k(z_k(L, t_i))}{\partial z_k} \right] \quad (52)$$

ix. Calculation of $\frac{\partial L_{I,i}}{\partial v_k}$

From Eq. (31) and (32):

$$\frac{\partial L_{I,i}}{\partial v_k} = 2R_{I,i}\sigma_k(z_k(x_i, 0)) + 2R_{\partial L,i}w_{2k} \frac{\partial \sigma_k(z_k(x_i, 0))}{\partial z_k} \quad (53)$$

x. Calculation of $\frac{\partial L_{I,i}}{\partial b_k}$

From Eq. (31) and (32):

$$\frac{\partial L_{I,i}}{\partial b_k} = 2R_{I,i}v_k \frac{\partial \sigma_k(z_k(x_i, 0))}{\partial z_k} + 2R_{\partial L,i}v_k w_{2k} \frac{\partial^2 \sigma_k(z_k(x_i, 0))}{\partial z_k^2} \quad (54)$$

xi. Calculation of $\frac{\partial L_{I,i}}{\partial w_{2k-1}}$

From Eq. (31) and (32):

$$\frac{\partial L_{I,i}}{\partial w_{2k-1}} = 2R_{I,i}v_k x_i \frac{\partial \sigma_k(z_k(x_i, 0))}{\partial z_k} + 2R_{\partial L,i}v_k w_{2k} x_i \frac{\partial^2 \sigma_k(z_k(x_i, 0))}{\partial z_k^2} \quad (55)$$

xii. Calculation of $\frac{\partial L_{I,i}}{\partial w_{2k}}$

From Eq. (31) and (32):

$$\frac{\partial L_{I,i}}{\partial w_{2k}} = 2R_{\partial L,i}v_k \frac{\partial \sigma_k(z_k(x_i, 0))}{\partial z_k} \quad (56)$$

4. Python Code

For Part A, as mentioned earlier, we write the code using standard Python with Numpy instead of PyTorch, as we want to code from scratch (without using existing deep learning libraries) the equations that we have derived above. The complete code is given in the Appendix, which can be run by those who are interested. However, we are not going to discuss the code line-by-line, but we will refer to snippets of the code that map to the equations we have derived.

The flowchart of the code is given in Figure 10.

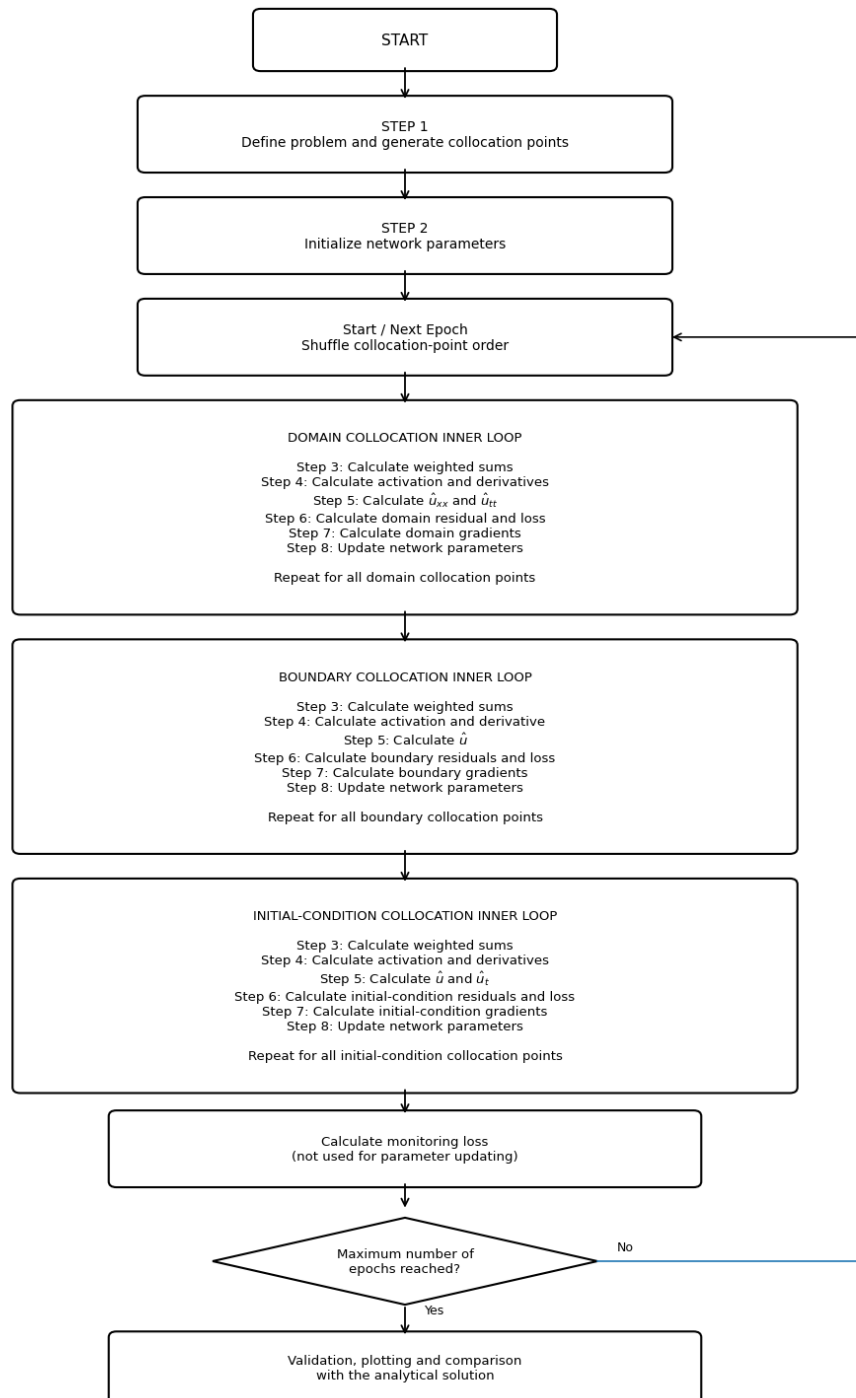


Figure 10: Training flowchart for the unscaled single-hidden-layer PINN

i. Upload Python libraries

For Part A, we just need Numpy and Matplotlib.

Code snippet for importing the libraries:

```
# MODULE 5 - PART A: PINN FROM SCRATCH FOR THE 1D WAVE EQUATION
import numpy as np
import matplotlib.pyplot as plt
```

ii. Step 1: Generate collocation point

We need to generate four sets of collocation points, in the domain, at the boundaries (left and right) and for the initial condition. As mentioned, since we are not using any scaling in Part A, we limit the length of the domain and the time to unity, thus $0 \leq x \leq 1$ and $0 \leq t \leq 1$. We also create the pairs of collocation points in the domain randomly.

In the code snippet below, the collocation points in the domain are randomly generated within the domain boundaries. For the boundary points, x is set to either 0 or 1, while t is running. For the initial-condition points, t is set to zero, while x is running.

Since Part A is unscaled, we set the domain length, L , the wave amplitude, A , and the wave propagation speed, c , all to unity.

Code snippet for Step 1:

```
# STEP 1: PROBLEM AND COLLOCATION POINTS
L, c, A = 1.0, 1.0, 1.0
n_D, n_B, n_I = 1000, 100, 100
x_min, x_max, t_min, t_max = 0.0, L, 0.0, 1.0
np.random.seed(1)
X_D = np.column_stack((np.random.uniform(x_min, x_max, n_D),
                        np.random.uniform(t_min, t_max, n_D)))
t_B = np.linspace(t_min, t_max, n_B)
X_B0 = np.column_stack((np.zeros(n_B), t_B))
X_BL = np.column_stack((L*np.ones(n_B), t_B))
X_I = np.column_stack((np.linspace(x_min, x_max, n_I), np.zeros(n_I)))
```

iii. Step 2: Initialize the network parameters

In this m -neuron single-hidden-layer PINN with two inputs, x_i , and t_i and a single output, we would have $2m$ weights, w , i.e.,

$$[w_1, w_2, \dots, w_{2m}, w_{2m-1}],$$

m output weights, v , i.e.,

$$[v_1, v_2, \dots, v_m],$$

and m biases, b , i.e.,

$$[b_1, b_2, \dots, b_m].$$

The weights w and v are randomly initialized using Xavier initialization, while the biases b are initially set to zero.

While the weights are initialized randomly, to ensure that we obtain the same initial values every time we run the code, we specify a random seed in the code (refer Step 1 snippet).

Code snippet for Step 2:

```
# STEP 2: INITIALIZE NETWORK PARAMETERS
m = 10
limit_w = np.sqrt(6.0/(2+m))
limit_v = np.sqrt(6.0/(m+1))
w = np.random.uniform(-limit_w, limit_w, 2*m)
b = np.zeros(m)
v = np.random.uniform(-limit_v, limit_v, m)
```

iv. Starting the training loops

Now that we have established the collocation points and initialized the parameters, we can begin the forward pass and the training process. For this, we need an outer loop that runs from Step 3 until the checking of convergence (which runs over the epochs), and inner loops for the respective collocation sets (domain, boundaries (left and right) and initial conditions), within which Steps 3 to 8 are carried out for each collocation point. The snippet for the outer loop is shown below. We use `random.permutation()` so that the order of the collocation points within the domain, boundary and initial-condition sets is randomly shuffled in each epoch. We also specify the learning rate at this stage.

Code snippet for setting up the outer loop

```
#SETTING UP THE TRAINING EXTERNAL LOOP|
eta = 0.0005
max_epochs = 2500
loss_history = []
for epoch in range(max_epochs):
    order_D = np.random.permutation(n_D)
    order_B = np.random.permutation(n_B)
    order_I = np.random.permutation(n_I)
    epoch_loss_D = epoch_loss_B = epoch_loss_I = 0.0
```

As mentioned, in the inner loop for each of the collocation sets, the following six steps will be carried out for each collocation point:

Step 3: Calculate the weighted sum

Step 4: Calculate the activation functions and their derivatives

Step 5: Calculate the network outputs and required network derivatives

Step 6: Calculate the residuals and losses

Step 7: Calculate the gradients

Step 8: Update the parameters

These steps will be evaluated accordingly for their respective collocation sets (domain, boundaries and initial conditions).

v. Domain collocation inner loop

The inner loop for domain collocation is given below.

Code snippet for setting up the inner loop for domain collocation process.

```
# DOMAIN COLLOCATION POINTS
for i in order_D:
    x_i, t_i = X_D[i]
```

- Step 3 for domain collocation

The snippet below shows the calculation of weighted sums for domain. It is the expanded form of Eq. (3), allowing for m neurons. Note, however, that in Python indexing starts from zero. Because of this, the subscript $2*k$ in the code corresponds to $2k - 1$ in our formulation, while $2*k+1$ corresponds to $2k$.

```
# STEP 3: Weighted sum
z = np.zeros(m)
for k in range(m):
    z[k] = w[2*k]*x_i + w[2*k+1]*t_i + b[k]#Eq. (3) for m neuron
```

- Step 4 for domain collocation

The snippet below shows the calculation of the activation function and its derivatives (up to the 3rd derivative is needed). It is the expanded form of Eq. (4), allowing for m neurons. Note that:

$$\sigma_k = \tanh(z_k)$$

$$\frac{\partial \sigma_k}{\partial z_k} = 1 - \sigma_k^2$$

$$\frac{\partial^2 \sigma_k}{\partial z_k^2} = -2\sigma_k(1 - \sigma_k^2) \quad (57)$$

$$\frac{\partial^3 \sigma_k}{\partial z_k^3} = -2(1 - \sigma_k^2)(1 - 3\sigma_k^2)$$

The derivatives in Eq. (57) are needed in the calculation of the gradients (i.e., Eqs. (45)-(56)) in Step 7.

```
# STEP 4: Activation function and derivatives
sigma = np.tanh(z) #Eq. (4) for m neuron
sigma_1 = 1.0-sigma**2
sigma_2 = -2.0*sigma*(1.0-sigma**2)
sigma_3 = -2.0*(1.0-sigma**2)*(1.0-3.0*sigma**2)
```

- Step 5 for domain collocation

The snippet below shows the calculation of the 2nd derivatives of the network with respect to x and t , respectively thus, giving NETWORK C and D (i.e., Eq. (19) and (20)).

```
# STEP 5: NETWORK C and NETWORK D
uxx = utt = 0.0
for k in range(m):
    uxx += v[k]*w[2*k]**2*sigma_2[k] # Eq. (19)
    utt += v[k]*w[2*k+1]**2*sigma_2[k] # Eq. (20)
```

- Step 6 for domain collocation

The snippet below shows the evaluation of the residual of the PDE (domain equation) and the corresponding square-error point loss (Eq. (23) and Eq. (24), respectively). epoch_loss is accumulated for monitoring purposes only (to plot the convergence graph).

```
# STEP 6: Domain residual and point loss
R_D = utt-c**2*uxx # Eq. (23)
L_D = R_D**2 # Eq. (24)
epoch_loss_D += L_D
```

- Step 7 for domain collocation

The snippet below shows the evaluation of the gradients (i.e., Eqs. (45) – (48)).

```
# STEP7: Domain gradients
gv, gb, gwx, gwt = np.zeros(m), np.zeros(m), np.zeros(m), np.zeros(m)
for k in range(m):
    wx, wt = w[2*k], w[2*k+1]
    gv[k] = 2*R_D*(wt**2-c**2*wx**2)*sigma_2[k] # Eq. (45)
    gb[k] = 2*R_D*v[k]*(wt**2-c**2*wx**2)*sigma_3[k] # Eq. (46)
    gwx[k] = 2*R_D*v[k]*(-2*c**2*wx*sigma_2[k]+x_i*(wt**2-c**2*wx**2)*sigma_3[k]) # Eq. (47)
    gwt[k] = 2*R_D*v[k]*(2*wt*sigma_2[k]+t_i*(wt**2-c**2*wx**2)*sigma_3[k]) # Eq. (48)
```

- Step 8 for domain collocation

This is the step where the parameters are updated based on the collocation point within the domain. The snippet shows the code for Eqs. (33) – (36).

```
# STEP 8: Parameter update
for k in range(m):
    v[k] = v[k] - eta*gv[k] # Eq. (33)
    b[k] = b[k] - eta*gb[k] # Eq. (34)
    w[2*k] = w[2*k] - eta*gwx[k] # Eq. (35)
    w[2*k+1] = w[2*k+1] - eta*gwt[k] # Eq. (36)
```

vi. Description for the rest of the code

In the code, the domain collocation inner loop is followed by the boundary inner loop and then by the initial-condition loop. Each of these loops has its own Step 3 to Step 8, based on the respective equations (and collocation points). There is no need to repeat the general description, as it would be similar to the one we already had for the domain collocation loop. It suffices to just highlight the equations coded in each loop as follows:

- Boundary inner loop coded equations
 - Step 3: Eq. (3) for m neuron
 - Step 4: Eq. (4) for m neuron
 - Step 5: Eq. (17)
 - Step 6: Eq. (27) and (28)
 - Step 7: Eq. (49) – (52)

Step 8: Eq. (37) – (40)

- Initial condition inner loop coded equations

Step 3: Eq. (3) for m neuron

Step 4: Eq. (4) for m neuron

Step 5: Eq. (17) and (18)

Step 6: Eq. (31) and (32)

Step 7: Eq. (53) – (56)

Step 8: Eq. (41) – (44)

Following the code for the collocation inner loops are the codes for plotting and verification of the results. The results are verified against the analytical solution, given as:

$$u(x, t) = A \sin\left(\frac{\pi x}{L}\right) \cos\frac{\pi c t}{L} \quad (58)$$

4.1 Plotting and verification of results

The results are shown in Figure 11 to 13 for the following combinations:

- $m=5$, epochs=1000
- $m=10$, epochs=2500
- $m=20$, epochs=5000
- $m=50$, epochs=10000

Figure 11 shows the convergence curves, Figure 12 shows the snapshots of the PINN predicted solution against the analytical solution at $t = 0.25$ and Figure 13 shows the PINN predicted solution “surface” against the analytical solution over x - t domain. As we can see, for all cases, the solution converges, with the results generally improving with the increase in the number of neurons (i.e., m) and iteration (i.e., epoch).

Modules for Machine and Deep Learning (Module 5: Part A – Physics-Informed Neural Network (PINN) from Scratch (Unscaled Single-Hidden-Layer Network))

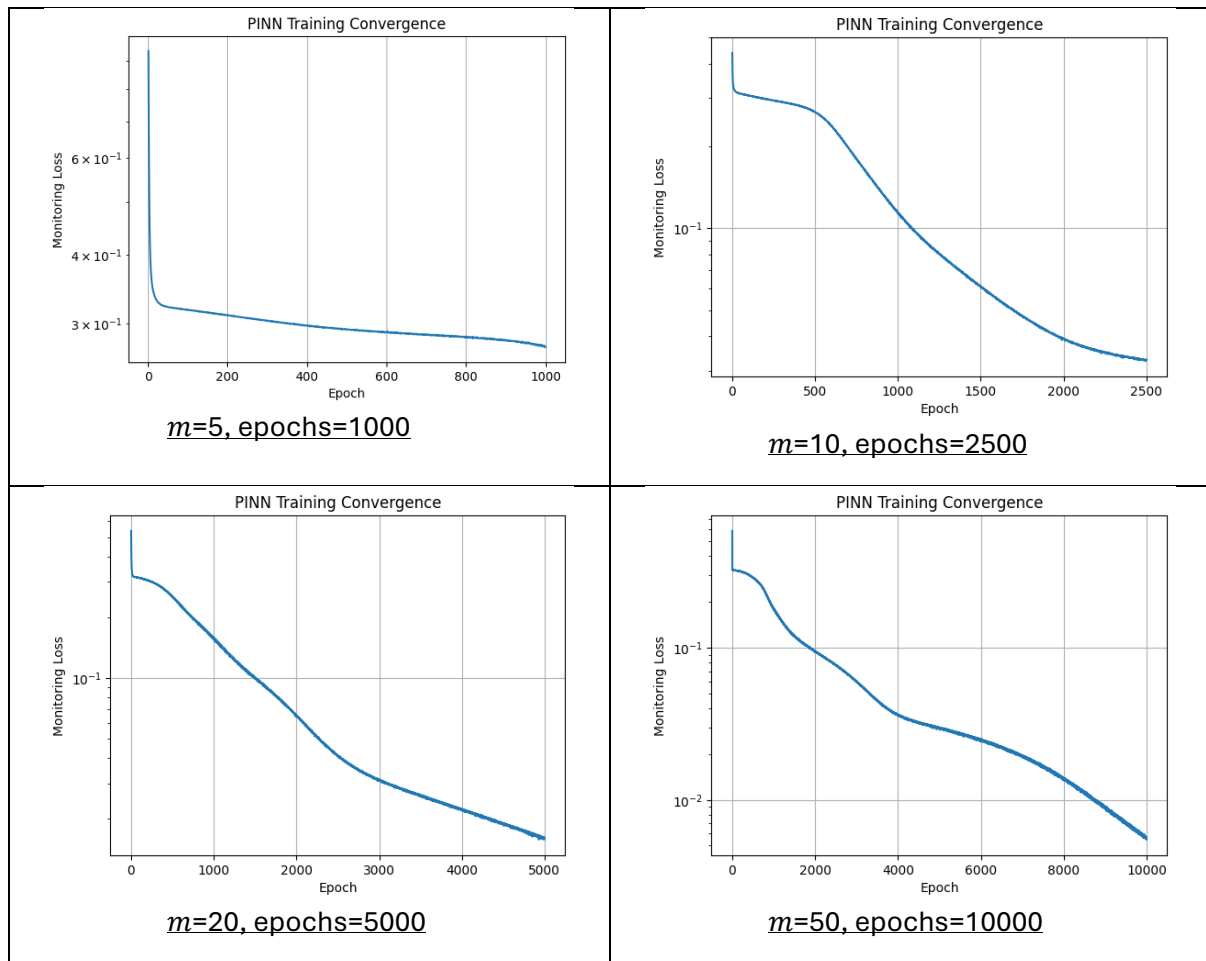


Figure 11: Plots of convergence curves for various combinations of m and epochs

Modules for Machine and Deep Learning (Module 5: Part A – Physics-Informed Neural Network (PINN) from Scratch (Unscaled Single-Hidden-Layer Network))

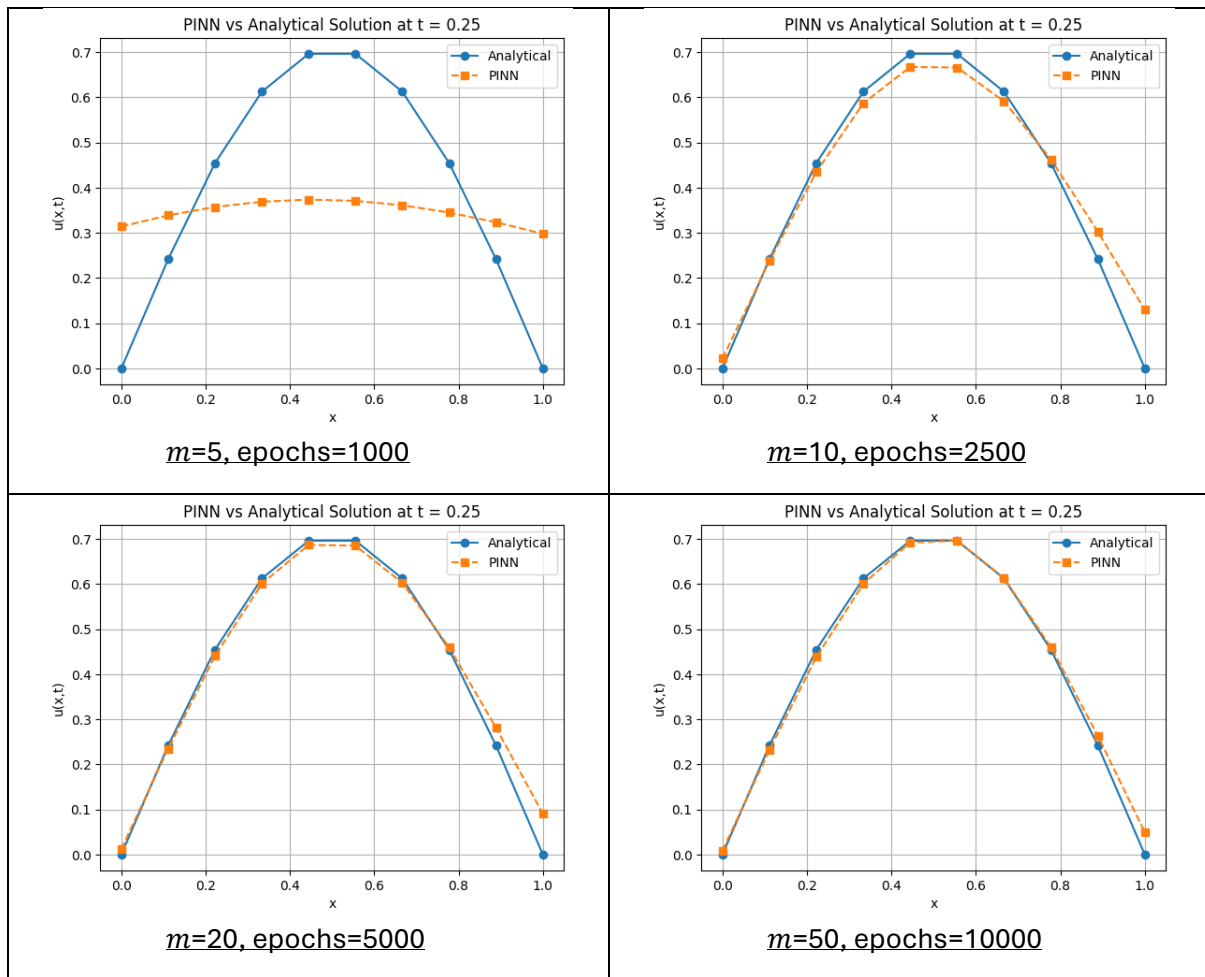


Figure 12: Snapshots of predicted and analytical solutions at $t=0.25$

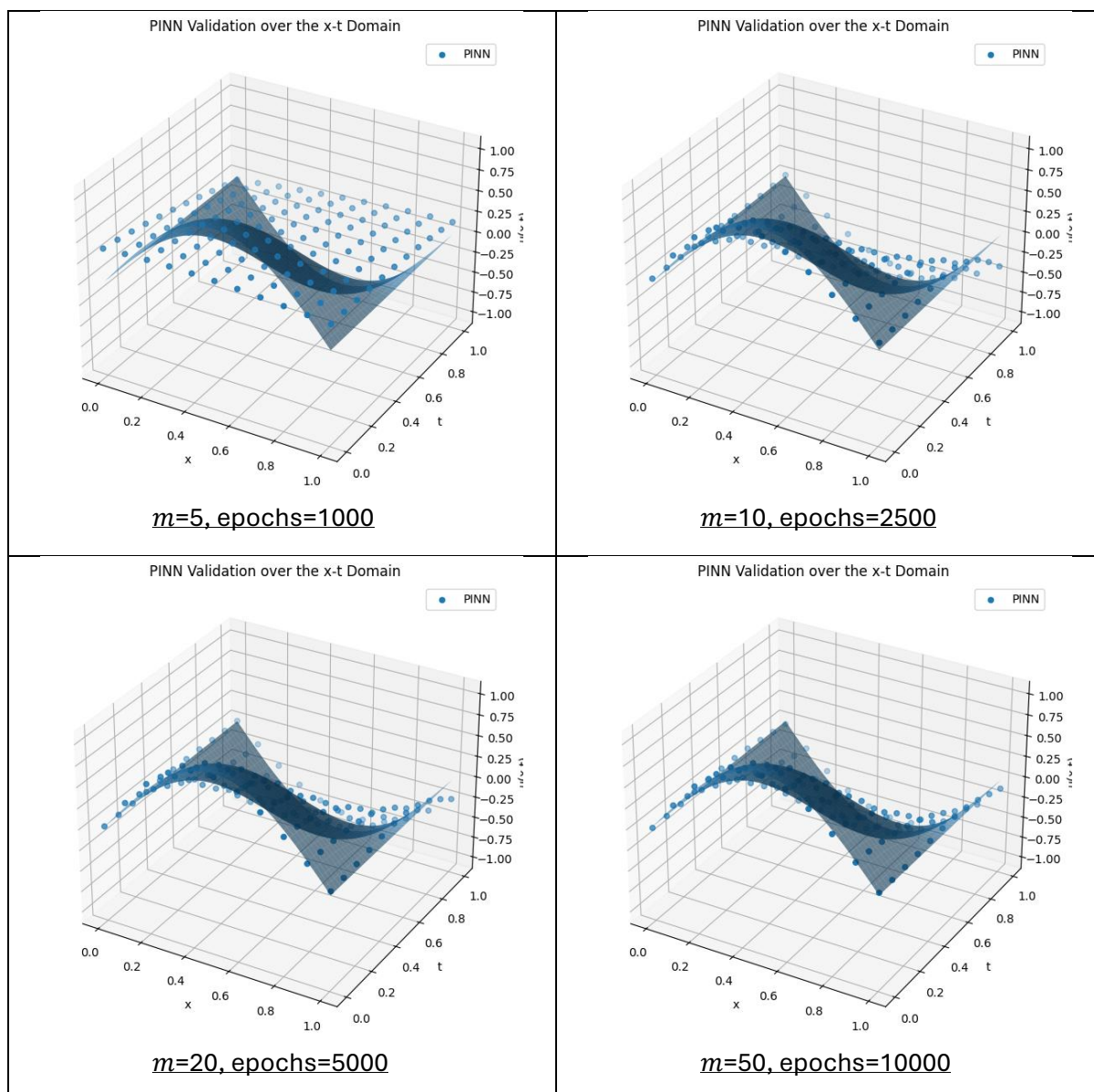


Figure 13: Predicted and analytical solution surfaces over the x - t domain

Appendix: Complete Python Code for the Unscaled Single-Hidden-Layer PINN

```
# MODULE 5 - PART A: PINN FROM SCRATCH FOR THE 1D WAVE EQUATION
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# STEP 1: PROBLEM AND COLLOCATION POINTS
```

```
L, c, A = 1.0, 1.0, 1.0
```

```
n_D, n_B, n_I = 1000, 100, 100
```

```
x_min, x_max, t_min, t_max = 0.0, L, 0.0, 1.0
```

```
np.random.seed(1)
```

```
X_D = np.column_stack((np.random.uniform(x_min, x_max, n_D),  
                        np.random.uniform(t_min, t_max, n_D)))
```

```
t_B = np.linspace(t_min, t_max, n_B)
```

```
X_B0 = np.column_stack((np.zeros(n_B), t_B))
```

```
X_BL = np.column_stack((L*np.ones(n_B), t_B))
```

```
X_I = np.column_stack((np.linspace(x_min, x_max, n_I), np.zeros(n_I)))
```

```
# STEP 2: INITIALIZE NETWORK PARAMETERS
```

```
m = 5
```

```
limit_w = np.sqrt(6.0/(2+m))
```

```
limit_v = np.sqrt(6.0/(m+1))
```

```
w = np.random.uniform(-limit_w, limit_w, 2*m)
```

```
b = np.zeros(m)
```

```
v = np.random.uniform(-limit_v, limit_v, m)
```

```
#SETTING UP THE TRAINING EXTERNAL LOOP
```

```
eta = 0.0005
```

```
max_epochs = 1000
```

```
loss_history = []
```

```
for epoch in range(max_epochs):
```

```
    order_D = np.random.permutation(n_D)
```

```
    order_B = np.random.permutation(n_B)
```

```
    order_I = np.random.permutation(n_I)
```

```
    epoch_loss_D = epoch_loss_B = epoch_loss_I = 0.0
```

```
    # DOMAIN COLLOCATION POINTS
```

```
    for i in order_D:
```

```
        x_i, t_i = X_D[i]
```

```
    # STEP 3: Weighted sum
```

Modules for Machine and Deep Learning (Module 5: Part A – Physics-Informed Neural Network (PINN) from Scratch (Unscaled Single-Hidden-Layer Network))

```

z = np.zeros(m)

for k in range(m):

    z[k] = w[2*k]*x_i + w[2*k+1]*t_i + b[k]#Eq. (3)

# STEP 4: Activation function and derivatives

sigma = np.tanh(z) #Eq. (4)

sigma_1 = 1.0-sigma**2

sigma_2 = -2.0*sigma*(1.0-sigma**2)

sigma_3 = -2.0*(1.0-sigma**2)*(1.0-3.0*sigma**2)

# STEP 5: NETWORK C and NETWORK D

uxx = utt = 0.0

for k in range(m):

    uxx += v[k]*w[2*k]**2*sigma_2[k]          # Eq. (19)

    utt += v[k]*w[2*k+1]**2*sigma_2[k]        # Eq. (20)

# STEP 6: Domain residual and point loss

R_D = utt-c**2*uxx                          # Eq. (23)

L_D = R_D**2                                # Eq. (24)

epoch_loss_D += L_D

# STEP7: Domain gradients

gv, gb, gwx, gwt = np.zeros(m),np.zeros(m),np.zeros(m),np.zeros(m)

for k in range(m):

    wx, wt = w[2*k], w[2*k+1]

    gv[k] = 2*R_D*(wt**2-c**2*wx**2)*sigma_2[k]          # Eq. (45)

    gb[k] = 2*R_D*v[k]*(wt**2-c**2*wx**2)*sigma_3[k]      # Eq. (46)

    gwx[k] = 2*R_D*v[k]*(-2*c**2*wx*sigma_2[k]+x_i*(wt**2-c**2*wx**2)*sigma_3[k]) # Eq. (47)

    gwt[k] = 2*R_D*v[k]*(2*wt*sigma_2[k]+t_i*(wt**2-c**2*wx**2)*sigma_3[k]) # Eq. (48)

# STEP 8: Parameter update

for k in range(m):

    v[k]= v[k]-eta*gv[k]          # Eq. (33)

    b[k]= b[k]-eta*gb[k]          # Eq. (34)

    w[2*k]= w[2*k]-eta*gwx[k]      # Eq. (35)

    w[2*k+1]= w[2*k+1]-eta*gwt[k]  # Eq. (36)

# BOUNDARY COLLOCATION POINTS

for i in order_B:

    t_i = X_B0[i,1]

    # STEP 3: Weighted sum at x=0 and x=L

```

Modules for Machine and Deep Learning (Module 5: Part A – Physics-Informed Neural Network (PINN) from Scratch (Unscaled Single-Hidden-Layer Network))

```

z0, zL = np.zeros(m), np.zeros(m)

for k in range(m):

    z0[k] = w[2*k+1]*t_i+b[k]

    zL[k] = w[2*k]*L+w[2*k+1]*t_i+b[k]

# STEP 4: Activation function and derivative
sigma0, sigmaL = np.tanh(z0), np.tanh(zL)
sigma0_1, sigmaL_1 = 1.0-sigma0**2, 1.0-sigmaL**2

# STEP 5: NETWORK A
u_B0 = u_BL = 0.0

for k in range(m):

    u_B0 += v[k]*sigma0[k]          # Eq. (17)
    u_BL += v[k]*sigmaL[k]         # Eq. (17)

# STEP 6: Boundary residuals and point loss
R_B0, R_BL = u_B0, u_BL          # Eq. (27)
L_B = R_B0**2+R_BL**2           # Eq. (28)
epoch_loss_B += L_B

# STEP 7: Boundary gradients
gv, gb, gwx, gwt = np.zeros(m), np.zeros(m), np.zeros(m), np.zeros(m)

for k in range(m):

    gv[k] = 2*R_B0*sigma0[k]+2*R_BL*sigmaL[k]          # Eq. (49)
    gb[k] = 2*v[k]*(R_B0*sigma0_1[k]+R_BL*sigmaL_1[k]) # Eq. (50)
    gwx[k] = 2*R_BL*v[k]*L*sigmaL_1[k]                 # Eq. (51)
    gwt[k] = 2*v[k]*t_i*(R_B0*sigma0_1[k]+R_BL*sigmaL_1[k]) # Eq. (52)

# STEP8 8: Parameter update
for k in range(m):

    v[k]=v[k]-eta*gv[k]          # Eq. (37)
    b[k]=b[k]-eta*gb[k]          # Eq. (38)
    w[2*k]=w[2*k]-eta*gwx[k]     # Eq. (39)
    w[2*k+1]=w[2*k+1]-eta*gwt[k] # Eq. (40)

# INITIAL-CONDITION COLLOCATION POINTS
for i in order_:

    x_i = X_[i,0]

    # STEP 3: Weighted sum at t=0

    z = np.zeros(m)

    for k in range(m):

```

Modules for Machine and Deep Learning (Module 5: Part A – Physics-Informed Neural Network (PINN) from Scratch (Unscaled Single-Hidden-Layer Network))

```

    z[k] = w[2*k]*x_i+b[k]

# STEP 4: Activation function and derivatives

sigma = np.tanh(z)

sigma_1 = 1.0-sigma**2

sigma_2 = -2.0*sigma*(1.0-sigma**2)

# STEP 5: NETWORK A and NETWORK B

u_l = ut_l = 0.0

for k in range(m):

    u_l += v[k]*sigma[k]          # Eq. (17)

    ut_l += v[k]*w[2*k+1]*sigma_1[k]    # Eq. (18)

# STEP 6: Initial-condition residuals and point loss

R_l = u_l-A*np.sin(np.pi*x_i/L)      # Eq. (31)

R_dl = ut_l                          # Eq. (31)

L_l = R_l**2+R_dl**2                  # Eq. (32)

epoch_loss_l += L_l

# STEP 7: Initial-condition gradients

gv, gb, gwx, gwt = np.zeros(m),np.zeros(m),np.zeros(m),np.zeros(m)

for k in range(m):

    wt = w[2*k+1]

    gv[k] = 2*R_l*sigma[k]+2*R_dl*wt*sigma_1[k]          # Eq. (53)

    gb[k] = 2*R_l*v[k]*sigma_1[k]+2*R_dl*v[k]*wt*sigma_2[k]    # Eq. (54)

    gwx[k] = 2*R_l*v[k]*x_i*sigma_1[k]+2*R_dl*v[k]*wt*x_i*sigma_2[k]    # Eq. (55)

    gwt[k] = 2*R_dl*v[k]*sigma_1[k]          # Eq. (56)

# STEP 8: Parameter update

for k in range(m):

    v[k]=v[k]-eta*gv[k]          # Eq. (41)

    b[k]=b[k]-eta*gb[k]          # Eq. (42)

    w[2*k]=w[2*k]-eta*gwx[k]      # Eq. (43)

    w[2*k+1]=w[2*k+1]-eta*gwt[k]  # Eq. (44)

# Monitoring only - NOT used for parameter updating

mean_LD = epoch_loss_D/n_D

mean_LB = epoch_loss_B/n_B

mean_LI = epoch_loss_l/n_l

monitoring_loss = mean_LD+mean_LB+mean_LI

loss_history.append(monitoring_loss)

```

Modules for Machine and Deep Learning (Module 5: Part A – Physics-Informed Neural Network (PINN) from Scratch (Unscaled Single-Hidden-Layer Network))

```

if (epoch+1)%200 == 0:
    print("Epoch =",epoch+1," Monitoring loss =",monitoring_loss)
# VALIDATION AGAINST ANALYTICAL SOLUTION
t_test = 0.25
x_test = np.linspace(0,L,10)
u_PINN, u_exact = np.zeros(len(x_test)), np.zeros(len(x_test))
for i in range(len(x_test)):
    z = np.zeros(m)
    for k in range(m):
        z[k] = w[2*k]*x_test[i]+w[2*k+1]*t_test+b[k]
    sigma = np.tanh(z)
    for k in range(m):
        u_PINN[i] += v[k]*sigma[k]          # NETWORK A, Eq. (17)
    u_exact[i] = A*np.sin(np.pi*x_test[i]/L)*np.cos(np.pi*c*t_test/L)
print("\nComparison at t =",t_test)
print(" x      PINN      Analytical      Absolute Error")
print("-----")
for i in range(len(x_test)):
    print(f"{x_test[i]:7.4f} {u_PINN[i]:10.6f} {u_exact[i]:10.6f} {abs(u_PINN[i]-u_exact[i]):12.6f}")
# 2D VALIDATION AND CONVERGENCE PLOTS
plt.figure()
plt.plot(x_test,u_exact,'o-',label='Analytical')
plt.plot(x_test,u_PINN,'s--',label='PINN')
plt.xlabel('x'); plt.ylabel('u(x,t)')
plt.title('PINN vs Analytical Solution at t = '+str(t_test))
plt.legend(); plt.grid(); plt.show()
plt.figure()
plt.semilogy(range(1,max_epochs+1),loss_history)
plt.xlabel('Epoch'); plt.ylabel('Monitoring Loss')
plt.title('PINN Training Convergence')
plt.grid(); plt.show()
# 3D VALIDATION OVER THE x-t DOMAIN
x_plot = np.linspace(0,L,41)
t_plot = np.linspace(t_min,t_max,41)
X_plot,T_plot = np.meshgrid(x_plot,t_plot)

```

Modules for Machine and Deep Learning (Module 5: Part A – Physics-Informed Neural Network (PINN) from Scratch (Unscaled Single-Hidden-Layer Network))

```

U_exact = A*np.sin(np.pi*X_plot/L)*np.cos(np.pi*c*T_plot/L)

U_PINN = np.zeros_like(X_plot)

for i in range(len(t_plot)):

    for j in range(len(x_plot)):

        z = np.zeros(m)

        for k in range(m):

            z[k] = w[2*k]*X_plot[i,j]+w[2*k+1]*T_plot[i,j]+b[k]

        sigma = np.tanh(z)

        for k in range(m):

            U_PINN[i,j] += v[k]*sigma[k]          # NETWORK A, Eq. (17)

fig = plt.figure(figsize=(9,7))

ax = fig.add_subplot(111,projection='3d')

ax.plot_surface(X_plot,T_plot,U_exact,alpha=0.6)

skip = 4

ax.scatter(X_plot[:,::skip],T_plot[:,::skip],

           U_PINN[:,::skip],marker='o',label='PINN')

ax.set_xlabel('x'); ax.set_ylabel('t'); ax.set_zlabel('u(x,t)')

ax.set_title('PINN Validation over the x-t Domain')

ax.legend()

plt.show()

```